

## Practical - 3 & 4

2520030030

U.Rohit Sai

### Programm - 1

#### Code:

```
rohitsai@MSI: ~/OS
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    pid_t pid;
    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());
    printf("Parent PPID: %d\n", getppid());
    pid = fork();
    if (pid < 0)
    {
        printf("Fork failed\n");
        return 1;
    }
    else if (pid == 0)
    {
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Child PPID: %d\n", getppid());
        printf("Child is running...\n");
        sleep(2);
        printf("Child process completed.\n");
    }
    else
    {
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
        printf("Parent is waiting for child...\n");
        wait(NULL);
        printf("Parent process completed.\n");
    }
    return 0;
}
```

Output :

```
rohitsai@MSI:~/OS$ nano ForkPid.c
rohitsai@MSI:~/OS$ gcc ForkPid.c -o ForkPid
rohitsai@MSI:~/OS$ ./ForkPid
Parent Process Started
Parent PID: 909
Parent PPID: 318

Parent Process
Parent PID: 909

Child PID: 910
Child Process
Parent is waiting for child...
Child PID: 910
Child PPID: 909
Child is running...
Child process completed.
Parent process completed.
```

## Programm - 2

### Code:

```
rohitsai@MSI: ~/OS
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    pid_t child1, child2;
    int status;
    child1 = fork();
    if (child1 == 0)
    {
        printf("Child 1 PID: %d\n", getpid());
        sleep(2);
        printf("Child 1 completed.\n");
        return 10;
    }
    child2 = fork();
    if (child2 == 0)
    {
        printf("Child 2 PID: %d\n", getpid());
        sleep(4);
        printf("Child 2 completed.\n");
        return 20;
    }
    printf("Parent PID: %d\n", getpid());
    printf("\nUsing wait()...\n");
    wait(&status);
    printf("One child completed using wait().\n");
    printf("\nUsing waitpid()...\n");
    waitpid(child2, &status, 0);
    printf("Child 2 completed using waitpid().\n");
    printf("Parent process completed.\n");
    return 0;
}
```

Output :

```
rohitsai@MSI:~/OS$ nano multi.c
rohitsai@MSI:~/OS$ gcc multi.c -o multi
rohitsai@MSI:~/OS$ ./multi
Child 1 PID: 945
Parent PID: 944

Using wait()...
Child 2 PID: 946
Child 1 completed.
One child completed using wait().

Using waitpid()...
Child 2 completed.
Child 2 completed using waitpid().
Parent process completed.
```